

```
#> 4   These days a chicken leg is a rare dish.      a chicken
#> 5       Rice is often served in round bowls.    <NA>    <NA>
#> 6       The juice of lemons makes fine punch.    <NA>    <NA>
#> # ... with 714 more rows
```

Like `str_extract()`, if you want all matches for each string, you'll need `str_match_all()`.

## Exercises

1. Find all words that come after a “number” like “one”, “two”, “three”, etc. Pull out both the number and the word.
2. Find all contractions. Separate out the pieces before and after the apostrophe.

## Replacing Matches

`str_replace()` and `str_replace_all()` allow you to replace matches with new strings. The simplest use is to replace a pattern with a fixed string:

```
x <- c("apple", "pear", "banana")
str_replace(x, "[aeiou]", "-")
#> [1] "-pple" "p-ar" "b-nana"
str_replace_all(x, "[aeiou]", "-")
#> [1] "-ppl-" "p--r" "b-n-n-"
```

With `str_replace_all()` you can perform multiple replacements by supplying a named vector:

```
x <- c("1 house", "2 cars", "3 people")
str_replace_all(x, c("1" = "one", "2" = "two", "3" = "three"))
#> [1] "one house" "two cars" "three people"
```

Instead of replacing with a fixed string you can use backreferences to insert components of the match. In the following code, I flip the order of the second and third words:

```
sentences %>%
  str_replace("([^\s]+) ([^\s]+) ([^\s]+)", "\\1 \\3 \\2") %>%
  head(5)
#> [1] "The canoe birch slid on the smooth planks."
#> [2] "Glue sheet the to the dark blue background."
#> [3] "It's to easy tell the depth of a well."
#> [4] "These a days chicken leg is a rare dish."
#> [5] "Rice often is served in round bowls."
```

## Exercises

1. Replace all forward slashes in a string with backslashes.
2. Implement a simple version of `str_to_lower()` using `replace_all()`.
3. Switch the first and last letters in words. Which of those strings are still words?

## Splitting

Use `str_split()` to split a string up into pieces. For example, we could split sentences into words:

```
sentences %>%
  head(5) %>%
  str_split(" ")
#> [[1]]
#> [1] "The"      "birch"    "canoe"    "slid"     "on"       "the"
#> [7] "smooth"   "planks."
#>
#> [[2]]
#> [1] "Glue"      "the"      "sheet"    "to"
#> [5] "the"      "dark"     "blue"     "background."
#>
#> [[3]]
#> [1] "It's"     "easy"     "to"       "tell"     "the"      "depth"   "of"
#> [8] "a"        "well."
#>
#> [[4]]
#> [1] "These"     "days"    "a"        "chicken"  "leg"      "is"
#> [7] "a"         "rare"     "dish."
#>
#> [[5]]
#> [1] "Rice"      "is"       "often"    "served"   "in"       "round"
#> [7] "bowls."
```

Because each component might contain a different number of pieces, this returns a list. If you're working with a length-1 vector, the easiest thing is to just extract the first element of the list:

```
"a|b|c|d" %>%
  str_split("\\|") %>%
  .[[1]]
#> [1] "a" "b" "c" "d"
```

Otherwise, like the other **stringr** functions that return a list, you can use `simplify = TRUE` to return a matrix:

```

sentences %>%
  head(5) %>%
  str_split(" ", simplify = TRUE)
#>      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
#> [1,] "The"    "birch"  "canoe"  "slid"    "on"    "the"    "smooth"
#> [2,] "Glue"    "the"    "sheet"  "to"      "the"    "dark"    "blue"
#> [3,] "It's"    "easy"   "to"     "tell"    "the"    "depth"  "of"
#> [4,] "These"   "days"  "a"      "chicken" "leg"    "is"     "a"
#> [5,] "Rice"    "is"     "often"  "served"  "in"     "round"  "bowls."
#>      [,8]      [,9]
#> [1,] "planks."  ""
#> [2,] "background." ""
#> [3,] "a"         "well."
#> [4,] "rare"      "dish."
#> [5,] ""        ""

```

You can also request a maximum number of pieces:

```

fields <- c("Name: Hadley", "Country: NZ", "Age: 35")
fields %>% str_split(":", n = 2, simplify = TRUE)
#>      [,1]      [,2]
#> [1,] "Name"    "Hadley"
#> [2,] "Country" "NZ"
#> [3,] "Age"     "35"

```

Instead of splitting up strings by patterns, you can also split up by character, line, sentence, and word boundary():

```

x <- "This is a sentence. This is another sentence."
str_view_all(x, boundary("word"))

```

```

This is a sentence. This is another sentence.

```

```

str_split(x, " ")[[1]]
#> [1] "This"    "is"      "a"       "sentence." ""
#> [6] "This"
#> [7] "is"      "another" "sentence."
str_split(x, boundary("word"))[[1]]
#> [1] "This"    "is"      "a"       "sentence" "This"
#> [6] "is"
#> [7] "another" "sentence"

```

## Exercises

1. Split up a string like "apples, pears, and bananas" into individual components.
2. Why is it better to split up by `boundary("word")` than `" "`?

3. What does splitting with an empty string ("" ) do? Experiment, and then read the documentation.

## Find Matches

`str_locate()` and `str_locate_all()` give you the starting and ending positions of each match. These are particularly useful when none of the other functions does exactly what you want. You can use `str_locate()` to find the matching pattern, and `str_sub()` to extract and/or modify them.

## Other Types of Pattern

When you use a pattern that's a string, it's automatically wrapped into a call to `regex()`:

```
# The regular call:
str_view(fruit, "nana")
# Is shorthand for
str_view(fruit, regex("nana"))
```

You can use the other arguments of `regex()` to control details of the match:

- `ignore_case = TRUE` allows characters to match either their uppercase or lowercase forms. This always uses the current locale:

```
bananas <- c("banana", "Banana", "BANANA")
str_view(bananas, "banana")
```

banana

Banana

BANANA

```
str_view(bananas, regex("banana", ignore_case = TRUE))
```

- `multiline = TRUE` allows `^` and `$` to match the start and end of each line rather than the start and end of the complete string:

```
x <- "Line 1\nLine 2\nLine 3"
str_extract_all(x, "^Line")[[1]]
#> [1] "Line"
str_extract_all(x, regex("^Line", multiline = TRUE))[[1]]
#> [1] "Line" "Line" "Line"
```

- `comments = TRUE` allows you to use comments and white space to make complex regular expressions more understandable. Spaces are ignored, as is everything after `#`. To match a literal space, you'll need to escape it: `"\\ "`.

```
phone <- regex("
  \\(?      # optional opening parens
  (\\d{3}) # area code
  [)]- ]?  # optional closing parens, dash, or space
  (\\d{3}) # another three numbers
  [ -]?   # optional space or dash
  (\\d{3}) # three more numbers
", comments = TRUE)

str_match("514-791-8141", phone)
#>      [,1]      [,2] [,3] [,4]
#> [1,] "514-791-814" "514" "791" "814"
```

- `dotall = TRUE` allows `.` to match everything, including `\n`.

There are three other functions you can use instead of `regex()`:

- `fixed()` matches exactly the specified sequence of bytes. It ignores all special regular expressions and operates at a very low level. This allows you to avoid complex escaping and can be much faster than regular expressions. The following microbenchmark shows that it's about 3x faster for a simple example:

```
microbenchmark::microbenchmark(
  fixed = str_detect(sentences, fixed("the")),
  regex = str_detect(sentences, "the"),
  times = 20
)
#> Unit: microseconds
#>   expr min  lq mean median   uq max neval cld
#> fixed 116 117 136   120 125 389   20  a
#> regex 333 337 346   338 342 467   20  b
```

Beware using `fixed()` with non-English data. It is problematic because there are often multiple ways of representing the same character. For example, there are two ways to define “á”: either as a single character or as an “a” plus an accent:

```
a1 <- "\u00e1"
a2 <- "a\u0301"
c(a1, a2)
#> [1] "á" "á"
```

```
a1 == a2
#> [1] FALSE
```

They render identically, but because they're defined differently, `fixed()` doesn't find a match. Instead, you can use `coll()`, defined next, to respect human character comparison rules:

```
str_detect(a1, fixed(a2))
#> [1] FALSE
str_detect(a1, coll(a2))
#> [1] TRUE
```

- `coll()` compares strings using standard *collation* rules. This is useful for doing case-insensitive matching. Note that `coll()` takes a `locale` parameter that controls which rules are used for comparing characters. Unfortunately different parts of the world use different rules!

```
# That means you also need to be aware of the difference
# when doing case-insensitive matches:
i <- c("I", "İ", "i", "ı")
i
#> [1] "I" "İ" "i" "ı"

str_subset(i, coll("i", ignore_case = TRUE))
#> [1] "I" "i"
str_subset(
  i,
  coll("i", ignore_case = TRUE, locale = "tr")
)
#> [1] "İ" "i"
```

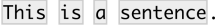
Both `fixed()` and `regex()` have `ignore_case` arguments, but they do not allow you to pick the locale: they always use the default locale. You can see what that is with the following code (more on **stringi** later):

```
stringi::stri_locale_info()
#> $Language
#> [1] "en"
#>
#> $Country
#> [1] "US"
#>
#> $Variant
#> [1] ""
#>
#> $Name
#> [1] "en_US"
```

The downside of `coll()` is speed; because the rules for recognizing which characters are the same are complicated, `coll()` is relatively slow compared to `regex()` and `fixed()`.

- As you saw with `str_split()`, you can use `boundary()` to match boundaries. You can also use it with the other functions:

```
x <- "This is a sentence."
str_view_all(x, boundary("word"))
```



```
str_extract_all(x, boundary("word"))
#> [[1]]
#> [1] "This"      "is"        "a"         "sentence"
```

## Exercises

1. How would you find all strings containing \ with `regex()` versus with `fixed()`?
2. What are the five most common words in sentences?

## Other Uses of Regular Expressions

There are two useful functions in base R that also use regular expressions:

- `apropos()` searches all objects available from the global environment. This is useful if you can't quite remember the name of the function:

```
apropos("replace")
#> [1] "%+replace%" "replace"      "replace_na"
#> [4] "str_replace" "str_replace_all" "str_replace_na"
#> [7] "theme_replace"
```

- `dir()` lists all the files in a directory. The `pattern` argument takes a regular expression and only returns filenames that match the pattern. For example, you can find all the R Markdown files in the current directory with:

```
head(dir(pattern = "\\\\.Rmd$"))
#> [1] "communicate-plots.Rmd" "communicate.Rmd"
```

```
#> [3] "datetimes.Rmd" "EDA.Rmd"  
#> [5] "explore.Rmd" "factors.Rmd"
```

(If you're more comfortable with "globs" like `*.Rmd`, you can convert them to regular expressions with `glob2rx()`).

## stringi

**stringr** is built on top of the **stringi** package. **stringr** is useful when you're learning because it exposes a minimal set of functions, which have been carefully picked to handle the most common string manipulation functions. **stringi**, on the other hand, is designed to be comprehensive. It contains almost every function you might ever need: **stringi** has 234 functions to **stringr**'s 42.

If you find yourself struggling to do something in **stringr**, it's worth taking a look at **stringi**. The packages work very similarly, so you should be able to translate your **stringr** knowledge in a natural way. The main difference is the prefix: `str_` versus `stri_`.

## Exercises

1. Find the **stringi** functions that:
  - a. Count the number of words.
  - b. Find duplicated strings.
  - c. Generate random text.
2. How do you control the language that `stri_sort()` uses for sorting?



---

# Factors with forcats

## Introduction

In R, factors are used to work with categorical variables, variables that have a fixed and known set of possible values. They are also useful when you want to display character vectors in a non-alphabetical order.

Historically, factors were much easier to work with than characters. As a result, many of the functions in base R automatically convert characters to factors. This means that factors often crop up in places where they're not actually helpful. Fortunately, you don't need to worry about that in the tidyverse, and can focus on situations where factors are genuinely useful.

For more historical context on factors, I recommend *stringsAsFactors: An unauthorized biography* by Roger Peng, and *stringsAsFactors = <sigh>* by Thomas Lumley.

## Prerequisites

To work with factors, we'll use the **forcats** package, which provides tools for dealing with *categorical* variables (and it's an anagram of factors!). It provides a wide range of helpers for working with factors. **forcats** is not part of the core tidyverse, so we need to load it explicitly.

```
library(tidyverse)
library(forcats)
```

# Creating Factors

Imagine that you have a variable that records month:

```
x1 <- c("Dec", "Apr", "Jan", "Mar")
```

Using a string to record this variable has two problems:

1. There are only twelve possible months, and there's nothing saving you from typos:

```
x2 <- c("Dec", "Apr", "Jam", "Mar")
```

2. It doesn't sort in a useful way:

```
sort(x1)
#> [1] "Apr" "Dec" "Jan" "Mar"
```

You can fix both of these problems with a factor. To create a factor you must start by creating a list of the valid *levels*:

```
month_levels <- c(
  "Jan", "Feb", "Mar", "Apr", "May", "Jun",
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
)
```

Now you can create a factor:

```
y1 <- factor(x1, levels = month_levels)
y1
#> [1] Dec Apr Jan Mar
#> Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
sort(y1)
#> [1] Jan Mar Apr Dec
#> Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

And any values not in the set will be silently converted to NA:

```
y2 <- factor(x2, levels = month_levels)
y2
#> [1] Dec Apr <NA> Mar
#> Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

If you want a want an error, you can use `readr::parse_factor()`:

```
y2 <- parse_factor(x2, levels = month_levels)
#> Warning: 1 parsing failure.
#> row col      expected actual
#> 3 -- value in level set    Jam
```

If you omit the levels, they'll be taken from the data in alphabetical order:

```
factor(x1)
#> [1] Dec Apr Jan Mar
#> Levels: Apr Dec Jan Mar
```

Sometimes you'd prefer that the order of the levels match the order of the first appearance in the data. You can do that when creating the factor by setting levels to `unique(x)`, or after the fact, with `fct_inorder()`:

```
f1 <- factor(x1, levels = unique(x1))
f1
#> [1] Dec Apr Jan Mar
#> Levels: Dec Apr Jan Mar

f2 <- x1 %>% factor() %>% fct_inorder()
f2
#> [1] Dec Apr Jan Mar
#> Levels: Dec Apr Jan Mar
```

If you ever need to access the set of valid levels directly, you can do so with `levels()`:

```
levels(f2)
#> [1] "Dec" "Apr" "Jan" "Mar"
```

## General Social Survey

For the rest of this chapter, we're going to focus on `forcats::gss_cat`. It's a sample of data from the **General Social Survey**, which is a long-running US survey conducted by the independent research organization NORC at the University of Chicago. The survey has thousands of questions, so in `gss_cat` I've selected a handful that will illustrate some common challenges you'll encounter when working with factors:

```
gss_cat
#> # A tibble: 21,483 × 9
#>   year marital age race rincome
#>   <int>   <fctr> <int> <fctr>   <fctr>
#> 1  2000 Never married  26 White $8000 to 9999
#> 2  2000 Divorced 48 White $8000 to 9999
#> 3  2000 Widowed 67 White Not applicable
#> 4  2000 Never married 39 White Not applicable
#> 5  2000 Divorced 25 White Not applicable
#> 6  2000 Married 25 White $20000 - 24999
#> # ... with 2.148e+04 more rows, and 4 more variables:
#> #   partyid <fctr>, relig <fctr>, denom <fctr>, tvhours <int>
```

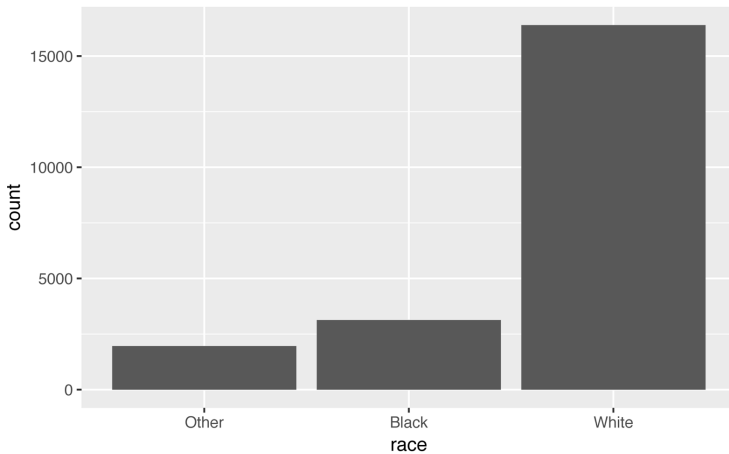
(Remember, since this dataset is provided by a package, you can get more information about the variables with `?gss_cat`.)

When factors are stored in a tibble, you can't see their levels so easily. One way to see them is with `count()`:

```
gss_cat %>%  
  count(race)  
#> # A tibble: 3 × 2  
#>   race      n  
#>   <fctr> <int>  
#> 1 Other  1959  
#> 2 Black 3129  
#> 3 White 16395
```

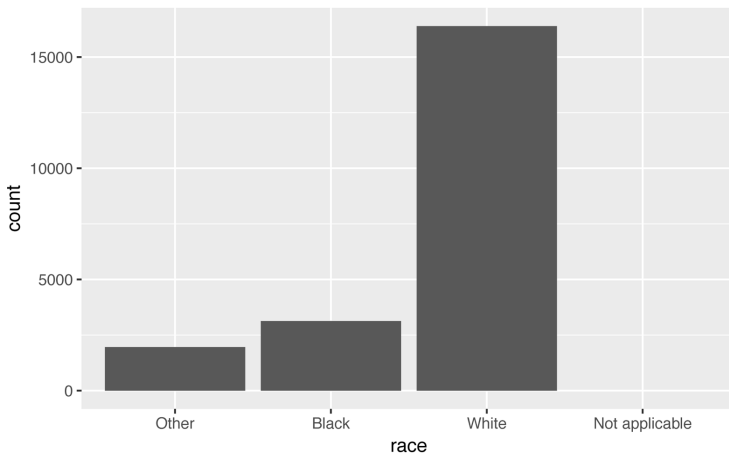
Or with a bar chart:

```
ggplot(gss_cat, aes(race)) +  
  geom_bar()
```



By default, **ggplot2** will drop levels that don't have any values. You can force them to display with:

```
ggplot(gss_cat, aes(race)) +  
  geom_bar() +  
  scale_x_discrete(drop = FALSE)
```



These levels represent valid values that simply did not occur in this dataset. Unfortunately, **dplyr** doesn't yet have a drop option, but it will in the future.

When working with factors, the two most common operations are changing the order of the levels, and changing the values of the levels. Those operations are described in the following sections.

## Exercises

1. Explore the distribution of `rincome` (reported income). What makes the default bar chart hard to understand? How could you improve the plot?
2. What is the most common `relig` in this survey? What's the most common `partyid`?
3. Which `relig` does `denom` (denomination) apply to? How can you find out with a table? How can you find out with a visualization?

## Modifying Factor Order

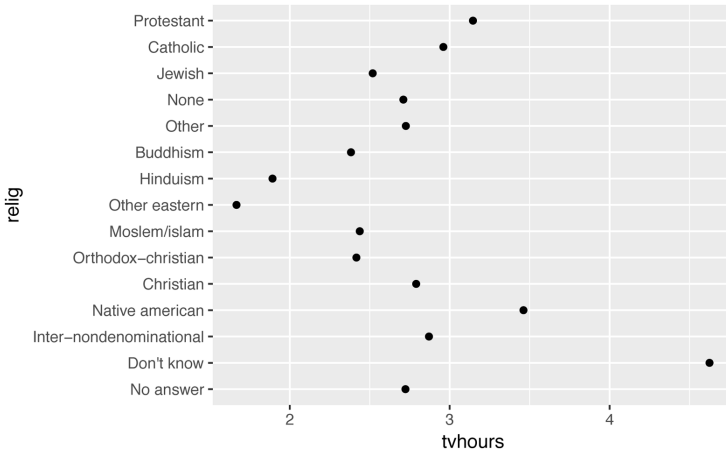
It's often useful to change the order of the factor levels in a visualization. For example, imagine you want to explore the average number of hours spent watching TV per day across religions:

```

relig <- gss_cat %>%
  group_by(relig) %>%
  summarize(
    age = mean(age, na.rm = TRUE),
    tvhours = mean(tvhours, na.rm = TRUE),
    n = n()
  )

ggplot(relig, aes(tvhours, relig)) + geom_point()

```



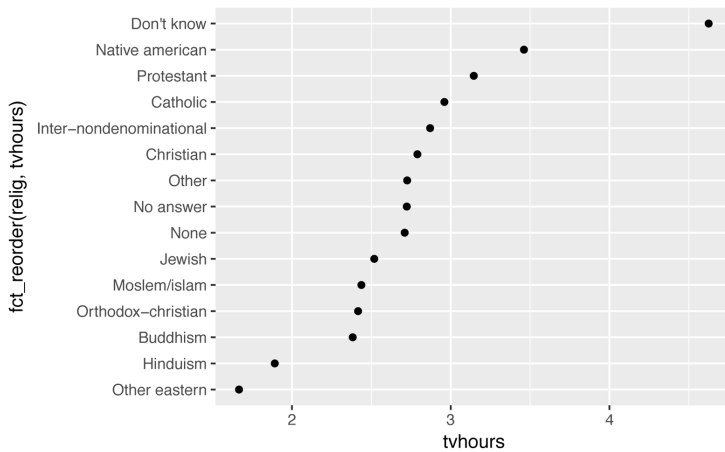
It is difficult to interpret this plot because there's no overall pattern. We can improve it by reordering the levels of `relig` using `fct_reorder()`. `fct_reorder()` takes three arguments:

- `f`, the factor whose levels you want to modify.
- `x`, a numeric vector that you want to use to reorder the levels.
- Optionally, `fun`, a function that's used if there are multiple values of `x` for each value of `f`. The default value is `median`.

```

ggplot(relig, aes(tvhours, fct_reorder(relig, tvhours))) +
  geom_point()

```



Reordering religion makes it much easier to see that people in the “Don’t know” category watch much more TV, and Hinduism and other Eastern religions watch much less.

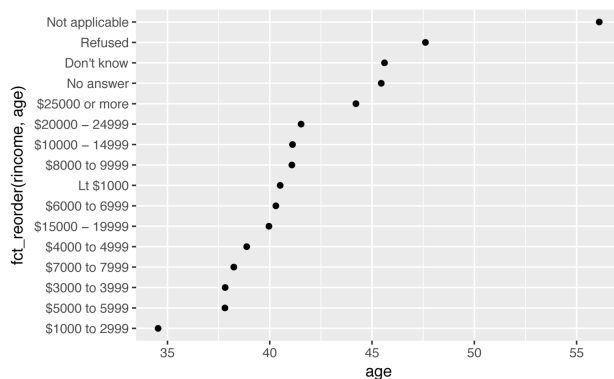
As you start making more complicated transformations, I’d recommend moving them out of `aes()` and into a separate `mutate()` step. For example, you could rewrite the preceding plot as:

```
relig %>%
  mutate(relig = fct_reorder(relig, tvhours)) %>%
  ggplot(aes(tvhours, relig)) +
    geom_point()
```

What if we create a similar plot looking at how average age varies across reported income level?

```
rincome <- gss_cat %>%
  group_by(rincome) %>%
  summarize(
    age = mean(age, na.rm = TRUE),
    tvhours = mean(tvhours, na.rm = TRUE),
    n = n()
  )

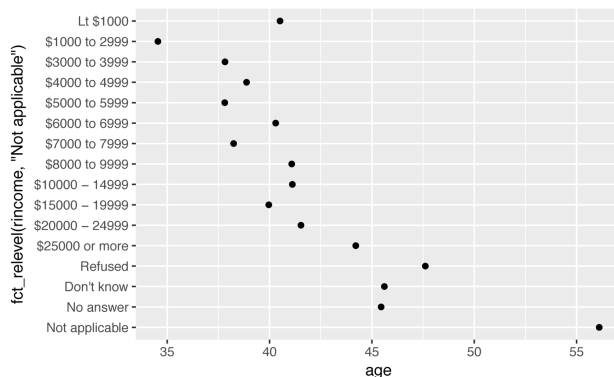
ggplot(
  rincome,
  aes(age, fct_reorder(rincome, age))
) + geom_point()
```



Here, arbitrarily reordering the levels isn't a good idea! That's because `rincome` already has a principled order that we shouldn't mess with. Reserve `fct_reorder()` for factors whose levels are arbitrarily ordered.

However, it does make sense to pull "Not applicable" to the front with the other special levels. You can use `fct_relevel()`. It takes a factor, `f`, and then any number of levels that you want to move to the front of the line:

```
ggplot(
  rincome,
  aes(age, fct_relevel(rincome, "Not applicable"))
) +
  geom_point()
```



Why do you think the average age for "Not applicable" is so high?

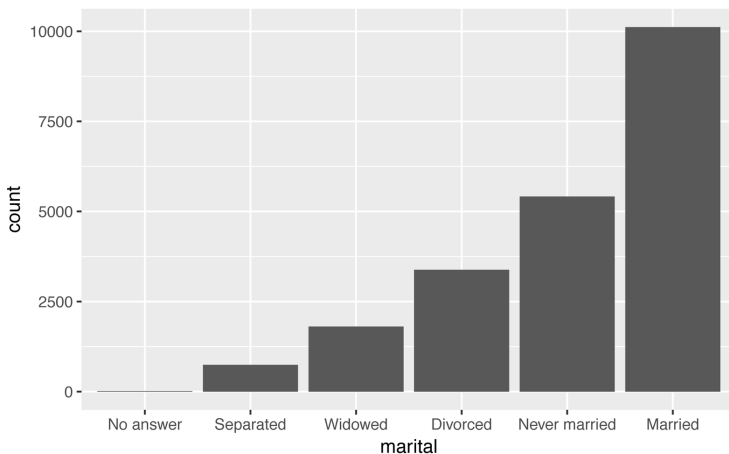


Another type of reordering is useful when you are coloring the lines on a plot. `fct_reorder2()` reorders the factor by the y values associated with the largest x values. This makes the plot easier to read because the line colors line up with the legend:

```
by_age <- gss_cat %>%  
  filter(!is.na(age)) %>%  
  group_by(age, marital) %>%  
  count() %>%  
  mutate(prop = n / sum(n))  
  
ggplot(by_age, aes(age, prop, color = marital)) +  
  geom_line(na.rm = TRUE)  
  
ggplot(  
  by_age,  
  aes(age, prop, color = fct_reorder2(marital, age, prop))  
) +  
  geom_line() +  
  labs(color = "marital")
```

Finally, for bar plots, you can use `fct_infreq()` to order levels in increasing frequency: this is the simplest type of reordering because it doesn't need any extra variables. You may want to combine with `fct_rev()`:

```
gss_cat %>%  
  mutate(marital = marital %>% fct_infreq() %>% fct_rev()) %>%  
  ggplot(aes(marital)) +  
  geom_bar()
```



## Exercises

1. There are some suspiciously high numbers in `tvhours`. Is the mean a good summary?
2. For each factor in `gss_cat` identify whether the order of the levels is arbitrary or principled.
3. Why did moving “Not applicable” to the front of the levels move it to the bottom of the plot?

## Modifying Factor Levels

More powerful than changing the orders of the levels is changing their values. This allows you to clarify labels for publication, and collapse levels for high-level displays. The most general and powerful tool is `fct_recode()`. It allows you to recode, or change, the value of each level. For example, take `gss_cat$partyid`:

```
gss_cat %>% count(partyid)
#> # A tibble: 10 × 2
#>   partyid      n
#>   <fctr> <int>
#> 1 No answer  154
#> 2 Don't know    1
#> 3 Other party 393
#> 4 Strong republican 2314
#> 5 Not str republican 3032
#> 6 Ind,near rep 1791
#> # ... with 4 more rows
```

The levels are terse and inconsistent. Let's tweak them to be longer and use a parallel construction:

```
gss_cat %>%
  mutate(partyid = fct_recode(partyid,
    "Republican, strong" = "Strong republican",
    "Republican, weak" = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak" = "Not str democrat",
    "Democrat, strong" = "Strong democrat"
  )) %>%
  count(partyid)
#> # A tibble: 10 × 2
#>   partyid      n
#>   <fctr> <int>
#> 1 No answer  154
```

```
#> 2          Don't know      1
#> 3          Other party    393
#> 4  Republican, strong  2314
#> 5  Republican, weak   3032
#> 6 Independent, near rep 1791
#> # ... with 4 more rows
```

`fct_recode()` will leave levels that aren't explicitly mentioned as is, and will warn you if you accidentally refer to a level that doesn't exist.

To combine groups, you can assign multiple old levels to the same new level:

```
gss_cat %>%
  mutate(partyid = fct_recode(partyid,
    "Republican, strong" = "Strong republican",
    "Republican, weak" = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak" = "Not str democrat",
    "Democrat, strong" = "Strong democrat",
    "Other" = "No answer",
    "Other" = "Don't know",
    "Other" = "Other party"
  )) %>%
  count(partyid)
#> # A tibble: 8 × 2
#>   partyid      n
#>   <fctr> <int>
#> 1 Other    548
#> 2 Republican, strong 2314
#> 3 Republican, weak 3032
#> 4 Independent, near rep 1791
#> 5 Independent    4119
#> 6 Independent, near dem 2499
#> # ... with 2 more rows
```

You must use this technique with care: if you group together categories that are truly different you will end up with misleading results.

If you want to collapse a lot of levels, `fct_collapse()` is a useful variant of `fct_recode()`. For each new variable, you can provide a vector of old levels:

```
gss_cat %>%
  mutate(partyid = fct_collapse(partyid,
    other = c("No answer", "Don't know", "Other party"),
    rep = c("Strong republican", "Not str republican"),
    ind = c("Ind,near rep", "Independent", "Ind,near dem"),
```

```

    dem = c("Not str democrat", "Strong democrat")
  }) %>%
  count(partyid)
#> # A tibble: 4 × 2
#>   partyid     n
#>   <fctr> <int>
#> 1   other   548
#> 2    rep  5346
#> 3    ind  8409
#> 4    dem  7180

```

Sometimes you just want to lump together all the small groups to make a plot or table simpler. That's the job of `fct_lump()`:

```

gss_cat %>%
  mutate(relig = fct_lump(relig)) %>%
  count(relig)
#> # A tibble: 2 × 2
#>   relig     n
#>   <fctr> <int>
#> 1 Protestant 10846
#> 2   Other  10637

```

The default behavior is to progressively lump together the smallest groups, ensuring that the aggregate is still the smallest group. In this case it's not very helpful: it is true that the majority of Americans in this survey are Protestant, but we've probably overcollapsed.

Instead, we can use the `n` parameter to specify how many groups (excluding other) we want to keep:

```

gss_cat %>%
  mutate(relig = fct_lump(relig, n = 10)) %>%
  count(relig, sort = TRUE) %>%
  print(n = Inf)
#> # A tibble: 10 × 2
#>   relig     n
#>   <fctr> <int>
#> 1 Protestant 10846
#> 2   Catholic  5124
#> 3     None   3523
#> 4   Christian  689
#> 5     Other   458
#> 6    Jewish   388
#> 7   Buddhism  147
#> 8 Inter-nondenominational 109
#> 9   Moslem/islam   104
#> 10 Orthodox-christian    95

```

## Exercises

1. How have the proportions of people identifying as Democrat, Republican, and Independent changed over time?
2. How could you collapse `income` into a small set of categories?



---

# Dates and Times with lubridate

## Introduction

This chapter will show you how to work with dates and times in R. At first glance, dates and times seem simple. You use them all the time in your regular life, and they don't seem to cause much confusion. However, the more you learn about dates and times, the more complicated they seem to get. To warm up, try these three seemingly simple questions:

- Does every year have 365 days?
- Does every day have 24 hours?
- Does every minute have 60 seconds?

I'm sure you know that not every year has 365 days, but do you know the full rule for determining if a year is a leap year? (It has three parts.) You might have remembered that many parts of the world use daylight saving time (DST), so that some days have 23 hours, and others have 25. You might not have known that some minutes have 61 seconds because every now and then leap seconds are added because the Earth's rotation is gradually slowing down.

Dates and times are hard because they have to reconcile two physical phenomena (the rotation of the Earth and its orbit around the sun) with a whole raft of geopolitical phenomena including months, time zones, and DST. This chapter won't teach you every last detail about dates and times, but it will give you a solid grounding of practical skills that will help you with common data analysis challenges.

## Prerequisites

This chapter will focus on the **lubridate** package, which makes it easier to work with dates and times in R. **lubridate** is not part of core tidyverse because you only need it when you're working with dates/times. We will also need **nycflights13** for practice data.

```
library(tidyverse)

library(lubridate)
library(nycflights13)
```

## Creating Date/Times

There are three types of date/time data that refer to an instant in time:

- A *date*. Tibbles print this as `<date>`.
- A *time* within a day. Tibbles print this as `<time>`.
- A *date-time* is a date plus a time: it uniquely identifies an instant in time (typically to the nearest second). Tibbles print this as `<dtm>`. Elsewhere in R these are called POSIXct, but I don't think that's a very useful name.

In this chapter we are only going to focus on dates and date-times as R doesn't have a native class for storing times. If you need one, you can use the **hms** package.

You should always use the simplest possible data type that works for your needs. That means if you can use a date instead of a date-time, you should. Date-times are substantially more complicated because of the need to handle time zones, which we'll come back to at the end of the chapter.

To get the current date or date-time you can use `today()` or `now()`:

```
today()
#> [1] "2016-10-10"
now()
#> [1] "2016-10-10 15:19:39 PDT"
```

Otherwise, there are three ways you're likely to create a date/time:



- From a string.
- From individual date-time components.
- From an existing date/time object.

They work as follows.

## From Strings

Date/time data often comes as strings. You’ve seen one approach to parsing strings into date-times in “[Dates, Date-Times, and Times](#)” on page 134. Another approach is to use the helpers provided by **lubridate**. They automatically work out the format once you specify the order of the component. To use them, identify the order in which year, month, and day appear in your dates, then arrange “y”, “m”, and “d” in the same order. That gives you the name of the **lubri-date** function that will parse your date. For example:

```
ymd("2017-01-31")
#> [1] "2017-01-31"
mdy("January 31st, 2017")
#> [1] "2017-01-31"
dmy("31-Jan-2017")
#> [1] "2017-01-31"
```

These functions also take unquoted numbers. This is the most concise way to create a single date/time object, as you might need when filtering date/time data. `ymd()` is short and unambiguous:

```
ymd(20170131)
#> [1] "2017-01-31"
```

`ymd()` and friends create dates. To create a date-time, add an underscore and one or more of “h”, “m”, and “s” to the name of the parsing function:

```
ymd_hms("2017-01-31 20:11:59")
#> [1] "2017-01-31 20:11:59 UTC"
mdy_hm("01/31/2017 08:01")
#> [1] "2017-01-31 08:01:00 UTC"
```

You can also force the creation of a date-time from a date by supplying a time zone:

```
ymd(20170131, tz = "UTC")
#> [1] "2017-01-31 UTC"
```

## From Individual Components

Instead of a single string, sometimes you'll have the individual components of the date-time spread across multiple columns. This is what we have in the flights data:

```
flights %>%
  select(year, month, day, hour, minute)
#> # A tibble: 336,776 × 5
#>   year month   day hour minute
#>   <int> <int> <int> <dbl> <dbl>
#> 1  2013     1     1     5     15
#> 2  2013     1     1     5     29
#> 3  2013     1     1     5     40
#> 4  2013     1     1     5     45
#> 5  2013     1     1     6      0
#> 6  2013     1     1     5     58
#> # ... with 3.368e+05 more rows
```

To create a date/time from this sort of input, use `make_date()` for dates, or `make_datetime()` for date-times:

```
flights %>%
  select(year, month, day, hour, minute) %>%
  mutate(
    departure = make_datetime(year, month, day, hour, minute)
  )
#> # A tibble: 336,776 × 6
#>   year month   day hour minute      departure
#>   <int> <int> <int> <dbl> <dbl>      <dtm>
#> 1  2013     1     1     5     15 2013-01-01 05:15:00
#> 2  2013     1     1     5     29 2013-01-01 05:29:00
#> 3  2013     1     1     5     40 2013-01-01 05:40:00
#> 4  2013     1     1     5     45 2013-01-01 05:45:00
#> 5  2013     1     1     6      0 2013-01-01 06:00:00
#> 6  2013     1     1     5     58 2013-01-01 05:58:00
#> # ... with 3.368e+05 more rows
```

Let's do the same thing for each of the four time columns in `flights`. The times are represented in a slightly odd format, so we use modulus arithmetic to pull out the hour and minute components. Once I've created the date-time variables, I focus in on the variables we'll explore in the rest of the chapter:

```
make_datetime_100 <- function(year, month, day, time) {
  make_datetime(year, month, day, time %/% 100, time %% 100)
}

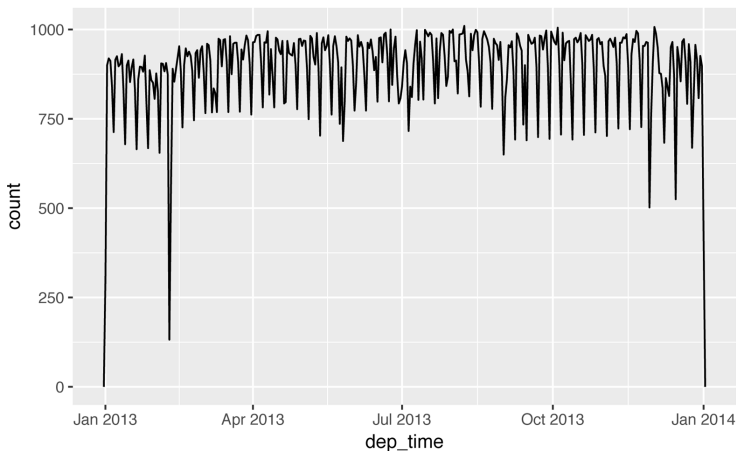
flights_dt <- flights %>%
  filter(!is.na(dep_time), !is.na(arr_time)) %>%
```

```
mutate(
  dep_time = make_datetime_100(year, month, day, dep_time),
  arr_time = make_datetime_100(year, month, day, arr_time),
  sched_dep_time = make_datetime_100(
    year, month, day, sched_dep_time
  ),
  sched_arr_time = make_datetime_100(
    year, month, day, sched_arr_time
  )
) %>%
select(origin, dest, ends_with("delay"), ends_with("time"))

flights_dt
#> # A tibble: 328,063 × 9
#>   origin dest dep_delay arr_delay      dep_time
#>   <chr> <chr>      <dbl>      <dbl>      <dtm>
#> 1   EWR  IAH         2         11 2013-01-01 05:17:00
#> 2   LGA  IAH         4         20 2013-01-01 05:33:00
#> 3   JFK  MIA         2         33 2013-01-01 05:42:00
#> 4   JFK  BQN        -1        -18 2013-01-01 05:44:00
#> 5   LGA  ATL        -6        -25 2013-01-01 05:54:00
#> 6   EWR  ORD        -4         12 2013-01-01 05:54:00
#> # ... with 3.281e+05 more rows, and 4 more variables:
#> #   sched_dep_time <dtm>, arr_time <dtm>,
#> #   sched_arr_time <dtm>, air_time <dbl>
```

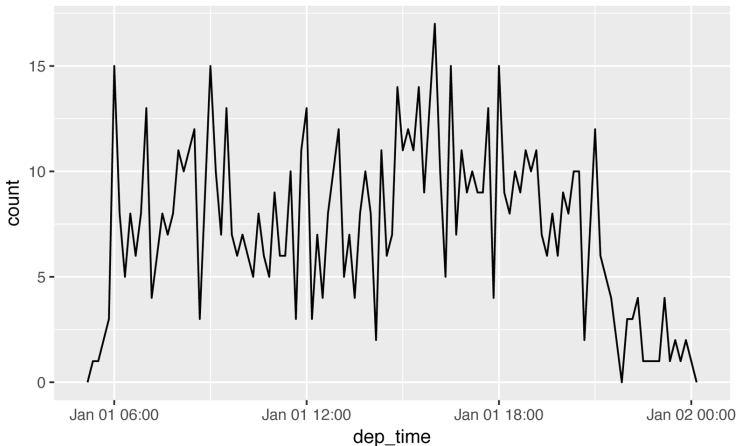
With this data, I can visualize the distribution of departure times across the year:

```
flights_dt %>%
  ggplot(aes(dep_time)) +
  geom_freqpoly(binwidth = 86400) # 86400 seconds = 1 day
```



Or within a single day:

```
flights_dt %>%  
  filter(dep_time < ymd(20130102)) %>%  
  ggplot(aes(dep_time)) +  
  geom_freqpoly(binwidth = 600) # 600 s = 10 minutes
```



Note that when you use date-times in a numeric context (like in a histogram), 1 means 1 second, so a binwidth of 86400 means one day. For dates, 1 means 1 day.

## From Other Types

You may want to switch between a date-time and a date. That's the job of `as_datetime()` and `as_date()`:

```
as_datetime(today())  
#> [1] "2016-10-10 UTC"  
as_date(now())  
#> [1] "2016-10-10"
```

Sometimes you'll get date/times as numeric offsets from the "Unix Epoch," 1970-01-01. If the offset is in seconds, use `as_datetime()`; if it's in days, use `as_date()`:

```
as_datetime(60 * 60 * 10)  
#> [1] "1970-01-01 10:00:00 UTC"  
as_date(365 * 10 + 2)  
#> [1] "1980-01-01"
```

## Exercises

1. What happens if you parse a string that contains invalid dates?  

```
ymd(c("2010-10-10", "bananas"))
```
2. What does the `tz` argument to `today()` do? Why is it important?
3. Use the appropriate **lubridate** function to parse each of the following dates:

```
d1 <- "January 1, 2010"
d2 <- "2015-Mar-07"
d3 <- "06-Jun-2017"
d4 <- c("August 19 (2015)", "July 1 (2015)")
d5 <- "12/30/14" # Dec 30, 2014
```

## Date-Time Components

Now that you know how to get date-time data into R's date-time data structures, let's explore what you can do with them. This section will focus on the accessor functions that let you get and set individual components. The next section will look at how arithmetic works with date-times.

### Getting Components

You can pull out individual parts of the date with the accessor functions `year()`, `month()`, `mday()` (day of the month), `yday()` (day of the year), `wday()` (day of the week), `hour()`, `minute()`, and `sec``ond()`:

```
datetime <- ymd_hms("2016-07-08 12:34:56")

year(datetime)
#> [1] 2016
month(datetime)
#> [1] 7
mday(datetime)
#> [1] 8

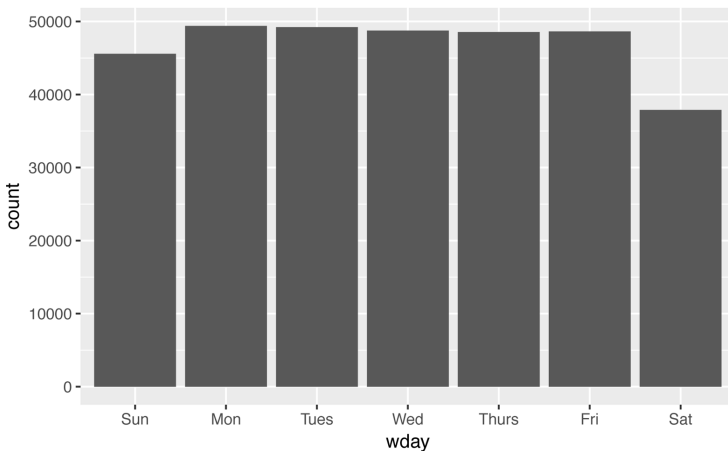
yday(datetime)
#> [1] 190
wday(datetime)
#> [1] 6
```

For `month()` and `wday()` you can set `label = TRUE` to return the abbreviated name of the month or day of the week. Set `abbr = FALSE` to return the full name:

```
month(datetime, label = TRUE)
#> [1] Jul
#> 12 Levels: Jan < Feb < Mar < Apr < May < Jun < ... < Dec
wday(datetime, label = TRUE, abbr = FALSE)
#> [1] Friday
#> 7 Levels: Sunday < Monday < Tuesday < ... < Saturday
```

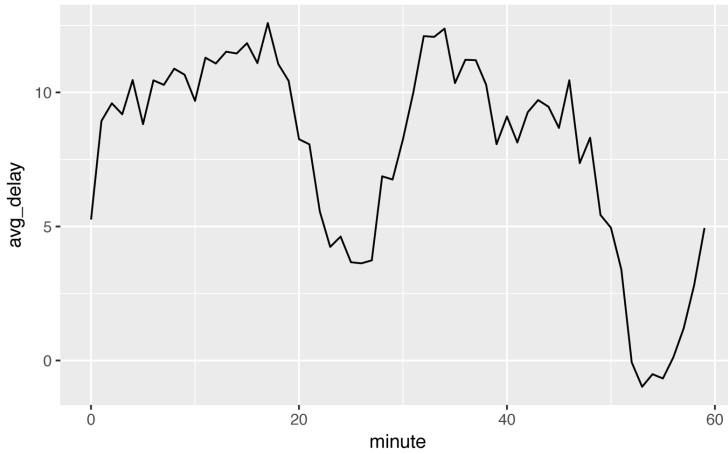
We can use `wday()` to see that more flights depart during the week than on the weekend:

```
flights_dt %>%
  mutate(wday = wday(dep_time, label = TRUE)) %>%
  ggplot(aes(x = wday)) +
  geom_bar()
```



There's an interesting pattern if we look at the average departure delay by minute within the hour. It looks like flights leaving in minutes 20–30 and 50–60 have much lower delays than the rest of the hour!

```
flights_dt %>%
  mutate(minute = minute(dep_time)) %>%
  group_by(minute) %>%
  summarize(
    avg_delay = mean(arr_delay, na.rm = TRUE),
    n = n()) %>%
  ggplot(aes(minute, avg_delay)) +
  geom_line()
```



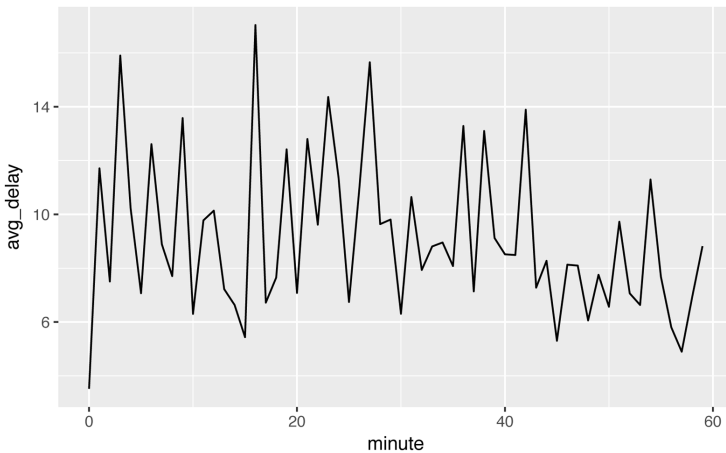
Interestingly, if we look at the *scheduled* departure time we don't see such a strong pattern:

```

sched_dep <- flights_dt %>%
  mutate(minute = minute(sched_dep_time)) %>%
  group_by(minute) %>%
  summarize(
    avg_delay = mean(arr_delay, na.rm = TRUE),
    n = n())

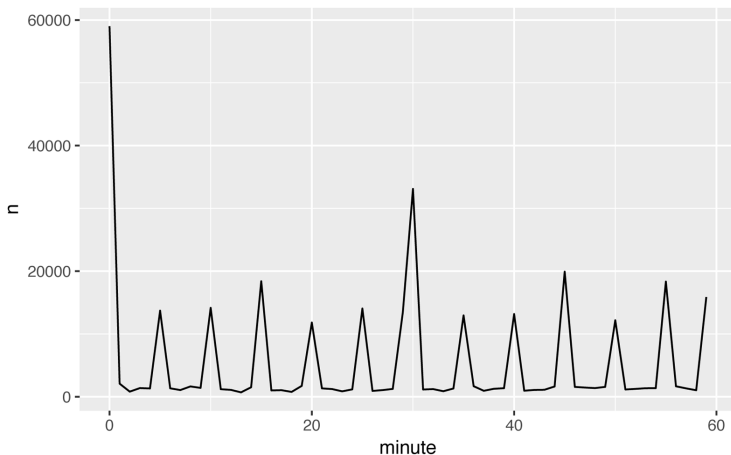
ggplot(sched_dep, aes(minute, avg_delay)) +
  geom_line()

```



So why do we see that pattern with the actual departure times? Well, like much data collected by humans, there's a strong bias toward flights leaving at “nice” departure times. Always be alert for this sort of pattern whenever you work with data that involves human judgment!

```
ggplot(sched_dep, aes(minute, n)) +  
  geom_line()
```

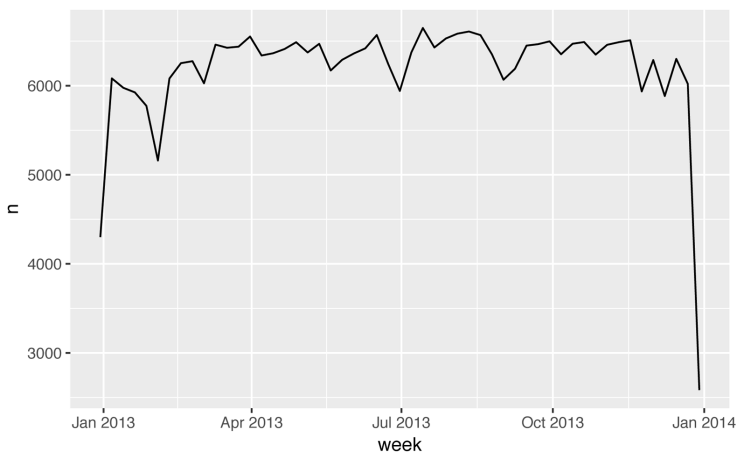


## Rounding

An alternative approach to plotting individual components is to round the date to a nearby unit of time, with `floor_date()`, `round_date()`, and `ceiling_date()`. Each `ceiling_date()` function takes a vector of dates to adjust and then the name of the unit to round down (floor), round up (ceiling), or round to. This, for example, allows us to plot the number of flights per week:

```
flights_dt %>%  
  count(week = floor_date(dep_time, "week")) %>%  
  ggplot(aes(week, n)) +  
    geom_line()
```





Computing the difference between a rounded and unrounded date can be particularly useful.

## Setting Components

You can also use each accessor function to set the components of a date/time:

```
(datetime <- ymd_hms("2016-07-08 12:34:56"))
#> [1] "2016-07-08 12:34:56 UTC"

year(datetime) <- 2020
datetime
#> [1] "2020-07-08 12:34:56 UTC"
month(datetime) <- 01
datetime
#> [1] "2020-01-08 12:34:56 UTC"
hour(datetime) <- hour(datetime) + 1
```

Alternatively, rather than modifying in place, you can create a new date-time with `update()`. This also allows you to set multiple values at once:

```
update(datetime, year = 2020, month = 2, mday = 2, hour = 2)
#> [1] "2020-02-02 02:34:56 UTC"
```

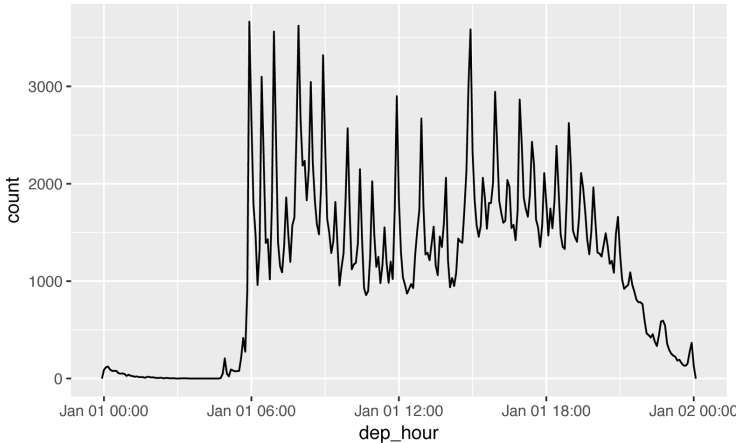
If values are too big, they will roll over:

```
ymd("2015-02-01") %>%
  update(mday = 30)
#> [1] "2015-03-02"
ymd("2015-02-01") %>%
```

```
update(hour = 400)
#> [1] "2015-02-17 16:00:00 UTC"
```

You can use `update()` to show the distribution of flights across the course of the day for every day of the year:

```
flights_dt %>%
  mutate(dep_hour = update(dep_time, yday = 1)) %>%
  ggplot(aes(dep_hour)) +
    geom_freqpoly(binwidth = 300)
```



Setting larger components of a date to a constant is a powerful technique that allows you to explore patterns in the smaller components.

## Exercises

1. How does the distribution of flight times within a day change over the course of the year?
2. Compare `dep_time`, `sched_dep_time`, and `dep_delay`. Are they consistent? Explain your findings.
3. Compare `air_time` with the duration between the departure and arrival. Explain your findings. (Hint: consider the location of the airport.)
4. How does the average delay time change over the course of a day? Should you use `dep_time` or `sched_dep_time`? Why?
5. On what day of the week should you leave if you want to minimize the chance of a delay?

6. What makes the distribution of `diamonds$carat` and `flights$sched_dep_time` similar?
7. Confirm my hypothesis that the early departures of flights in minutes 20–30 and 50–60 are caused by scheduled flights that leave early. Hint: create a binary variable that tells you whether or not a flight was delayed.

## Time Spans

Next you'll learn about how arithmetic with dates works, including subtraction, addition, and division. Along the way, you'll learn about three important classes that represent time spans:

- *Durations*, which represent an exact number of seconds.
- *Periods*, which represent human units like weeks and months.
- *Intervals*, which represent a starting and ending point.

### Durations

In R, when you subtract two dates, you get a `difftime` object:

```
# How old is Hadley?
h_age <- today() - ymd(19791014)
h_age
#> Time difference of 13511 days
```

A `difftime` class object records a time span of seconds, minutes, hours, days, or weeks. This ambiguity can make `diffetimes` a little painful to work with, so **lubridate** provides an alternative that always uses seconds—the *duration*:

```
as.duration(h_age)
#> [1] "1167350400s (~36.99 years)"
```

Durations come with a bunch of convenient constructors:

```
dseconds(15)
#> [1] "15s"
dminutes(10)
#> [1] "600s (~10 minutes)"
dhours(c(12, 24))
#> [1] "43200s (~12 hours)" "86400s (~1 days)"
ddays(0:5)
#> [1] "0s" "86400s (~1 days)"
#> [3] "172800s (~2 days)" "259200s (~3 days)"
```

```
#> [5] "345600s (~4 days)" "432000s (~5 days)"
dweeks(3)
#> [1] "1814400s (~3 weeks)"
dyears(1)
#> [1] "31536000s (~52.14 weeks)"
```

Durations always record the time span in seconds. Larger units are created by converting minutes, hours, days, weeks, and years to seconds at the standard rate (60 seconds in a minute, 60 minutes in an hour, 24 hours in a day, 7 days in a week, 365 days in a year).

You can add and multiply durations:

```
2 * dyears(1)
#> [1] "63072000s (~2 years)"
dyears(1) + dweeks(12) + dhours(15)
#> [1] "38847600s (~1.23 years)"
```

You can add and subtract durations to and from days:

```
tomorrow <- today() + ddays(1)
last_year <- today() - dyears(1)
```

However, because durations represent an exact number of seconds, sometimes you might get an unexpected result:

```
one_pm <- ymd_hms(
  "2016-03-12 13:00:00",
  tz = "America/New_York"
)

one_pm
#> [1] "2016-03-12 13:00:00 EST"
one_pm + ddays(1)
#> [1] "2016-03-13 14:00:00 EDT"
```

Why is one day after 1 p.m. on March 12, 2 p.m. on March 13?! If you look carefully at the date you might also notice that the time zones have changed. Because of DST, March 12 only has 23 hours, so if we add a full day's worth of seconds we end up with a different time.

## Periods

To solve this problem, **lubridate** provides *periods*. Periods are time spans but don't have a fixed length in seconds; instead they work with "human" times, like days and months. That allows them to work in a more intuitive way:

```
one_pm
#> [1] "2016-03-12 13:00:00 EST"
one_pm + days(1)
#> [1] "2016-03-13 13:00:00 EDT"
```

Like durations, periods can be created with a number of friendly constructor functions:

```
seconds(15)
#> [1] "15S"
minutes(10)
#> [1] "10M 0S"
hours(c(12, 24))
#> [1] "12H 0M 0S" "24H 0M 0S"
days(7)
#> [1] "7d 0H 0M 0S"
months(1:6)
#> [1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S"
#> [4] "4m 0d 0H 0M 0S" "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
weeks(3)
#> [1] "21d 0H 0M 0S"
years(1)
#> [1] "1y 0m 0d 0H 0M 0S"
```

You can add and multiply periods:

```
10 * (months(6) + days(1))
#> [1] "60m 10d 0H 0M 0S"
days(50) + hours(25) + minutes(2)
#> [1] "50d 25H 2M 0S"
```

And of course, add them to dates. Compared to durations, periods are more likely to do what you expect:

```
# A leap year
ymd("2016-01-01") + dyears(1)
#> [1] "2016-12-31"
ymd("2016-01-01") + years(1)
#> [1] "2017-01-01"

# Daylight Savings Time
one_pm + ddays(1)
#> [1] "2016-03-13 14:00:00 EDT"
one_pm + days(1)
#> [1] "2016-03-13 13:00:00 EDT"
```

Let's use periods to fix an oddity related to our flight dates. Some planes appear to have arrived at their destination *before* they departed from New York City:

```
flights_dt %>%
  filter(arr_time < dep_time)
```

```
#> # A tibble: 10,633 × 9
#>   origin dest dep_delay arr_delay dep_time
#>   <chr> <chr>     <dbl>     <dbl>     <dtm>
#> 1   EWR  BQN         9        -4 2013-01-01 19:29:00
#> 2   JFK  DFW        59         NA 2013-01-01 19:39:00
#> 3   EWR  TPA        -2         9 2013-01-01 20:58:00
#> 4   EWR  SJU        -6       -12 2013-01-01 21:02:00
#> 5   EWR  SFO        11       -14 2013-01-01 21:08:00
#> 6   LGA  FLL       -10        -2 2013-01-01 21:20:00
#> # ... with 1.063e+04 more rows, and 4 more variables:
#> #   sched_dep_time <dtm>, arr_time <dtm>,
#> #   sched_arr_time <dtm>, air_time <dbl>
```

These are overnight flights. We used the same date information for both the departure and the arrival times, but these flights arrived on the following day. We can fix this by adding `days(1)` to the arrival time of each overnight flight:

```
flights_dt <- flights_dt %>%
  mutate(
    overnight = arr_time < dep_time,
    arr_time = arr_time + days(overnight * 1),
    sched_arr_time = sched_arr_time + days(overnight * 1)
  )
```

Now all of our flights obey the laws of physics:

```
flights_dt %>%
  filter(overnight, arr_time < dep_time)
#> # A tibble: 0 × 10
#> # ... with 10 variables: origin <chr>, dest <chr>,
#> #   dep_delay <dbl>, arr_delay <dbl>, dep_time <dtm>,
#> #   sched_dep_time <dtm>, arr_time <dtm>,
#> #   sched_arr_time <dtm>, air_time <dbl>, overnight <lgl>
```

## Intervals

It's obvious what `dyears(1) / ddays(365)` should return: one, because durations are always represented by a number of seconds, and a duration of a year is defined as 365 days' worth of seconds.

What should `years(1) / days(1)` return? Well, if the year was 2015 it should return 365, but if it was 2016, it should return 366! There's not quite enough information for **lubridate** to give a single clear answer. What it does instead is give an estimate, with a warning:

```
years(1) / days(1)
#> estimate only: convert to intervals for accuracy
#> [1] 365
```

If you want a more accurate measurement, you'll have to use an *interval*. An interval is a duration with a starting point; that makes it precise so you can determine exactly how long it is:

```
next_year <- today() + years(1)
(today() %--% next_year) / ddays(1)
#> [1] 365
```

To find out how many periods fall into an interval, you need to use integer division:

```
(today() %--% next_year) %/% days(1)
#> [1] 365
```

## Summary

How do you pick between duration, periods, and intervals? As always, pick the simplest data structure that solves your problem. If you only care about physical time, use a duration; if you need to add human times, use a period; if you need to figure out how long a span is in human units, use an interval.

Figure 13-1 summarizes permitted arithmetic operations between the different data types.

|           | date |   |  | date time |   |  | duration |   |   | period |   |   | interval |   |   | number |   |   |
|-----------|------|---|--|-----------|---|--|----------|---|---|--------|---|---|----------|---|---|--------|---|---|
| date      | -    |   |  |           |   |  | -        | + |   | -      | + |   |          |   |   | -      | + |   |
| date time |      |   |  | -         |   |  | -        | + |   | -      | + |   |          |   |   | -      | + |   |
| duration  | -    | + |  | -         | + |  | -        | + | / |        |   |   |          |   |   | -      | + | × |
| period    | -    | + |  | -         | + |  |          |   |   | -      | + |   |          |   |   | -      | + | × |
| interval  |      |   |  |           |   |  |          |   | / |        |   | / |          |   |   |        |   |   |
| number    | -    | + |  | -         | + |  | -        | + | × | -      | + | × | -        | + | × | -      | + | × |

Figure 13-1. The allowed arithmetic operations between pairs of date/time classes

## Exercises

1. Why is there `months()` but no `dmonths()`?
2. Explain `days(overnight * 1)` to someone who has just started learning R. How does it work?

3. Create a vector of dates giving the first day of every month in 2015. Create a vector of dates giving the first day of every month in the *current* year.
4. Write a function that, given your birthday (as a date), returns how old you are in years.
5. Why can't `(today() %--% (today() + years(1)) / months(1))` work?

## Time Zones

Time zones are an enormously complicated topic because of their interaction with geopolitical entities. Fortunately we don't need to dig into all the details as they're not all that important for data analysis, but there are a few challenges we'll need to tackle head on.

The first challenge is that everyday names of time zones tend to be ambiguous. For example, if you're American you're probably familiar with EST, or Eastern Standard Time. However, both Australia and Canada also have EST! To avoid confusion, R uses the international standard IANA time zones. These use a consistent naming scheme with "/", typically in the form "<continent>/<city>" (there are a few exceptions because not every country lies on a continent). Examples include "America/New\_York," "Europe/Paris," and "Pacific/Auckland."

You might wonder why the time zone uses a city, when typically you think of time zones as associated with a country or region within a country. This is because the IANA database has to record decades' worth of time zone rules. In the course of decades, countries change names (or break apart) fairly frequently, but city names tend to stay the same. Another problem is that name needs to reflect not only to the current behavior, but also the complete history. For example, there are time zones for both "America/New\_York" and "America/Detroit." These cities both currently use Eastern Standard Time but in 1969–1972, Michigan (the state in which Detroit is located) did not follow DST, so it needs a different name. It's worth reading the raw time zone database (available at <http://www.iana.org/time-zones>) just to read some of these stories!

You can find out what R thinks your current time zone is with `Sys.timezone()`:



```
Sys.timezone()
#> [1] "America/Los_Angeles"
```

(If R doesn't know, you'll get an NA.)

And see the complete list of all time zone names with `OlsonNames()`:

```
length(OlsonNames())
#> [1] 589
head(OlsonNames())
#> [1] "Africa/Abidjan"      "Africa/Accra"
#> [3] "Africa/Addis_Ababa"  "Africa/Algiers"
#> [5] "Africa/Asmara"       "Africa/Asmera"
```

In R, the time zone is an attribute of the date-time that only controls printing. For example, these three objects represent the same instant in time:

```
(x1 <- ymd_hms("2015-06-01 12:00:00", tz = "America/New_York"))
#> [1] "2015-06-01 12:00:00 EDT"
(x2 <- ymd_hms("2015-06-01 18:00:00", tz = "Europe/Copenhagen"))
#> [1] "2015-06-01 18:00:00 CEST"
(x3 <- ymd_hms("2015-06-02 04:00:00", tz = "Pacific/Auckland"))
#> [1] "2015-06-02 04:00:00 NZST"
```

You can verify that they're the same time using subtraction:

```
x1 - x2
#> Time difference of 0 secs
x1 - x3
#> Time difference of 0 secs
```

Unless otherwise specified, **lubridate** always uses UTC. UTC (Coordinated Universal Time) is the standard time zone used by the scientific community and is roughly equivalent to its predecessor GMT (Greenwich Mean Time). It does not have DST, which makes it a convenient representation for computation. Operations that combine date-times, like `c()`, will often drop the time zone. In that case, the date-times will display in your local time zone:

```
x4 <- c(x1, x2, x3)
x4
#> [1] "2015-06-01 09:00:00 PDT" "2015-06-01 09:00:00 PDT"
#> [3] "2015-06-01 09:00:00 PDT"
```

You can change the time zone in two ways:

- Keep the instant in time the same, and change how it's displayed. Use this when the instant is correct, but you want a more natural display:

```
x4a <- with_tz(x4, tzone = "Australia/Lord_Howe")
x4a
#> [1] "2015-06-02 02:30:00 LHST"
#> [2] "2015-06-02 02:30:00 LHST"
#> [3] "2015-06-02 02:30:00 LHST"
x4a - x4
#> Time differences in secs
#> [1] 0 0 0
```

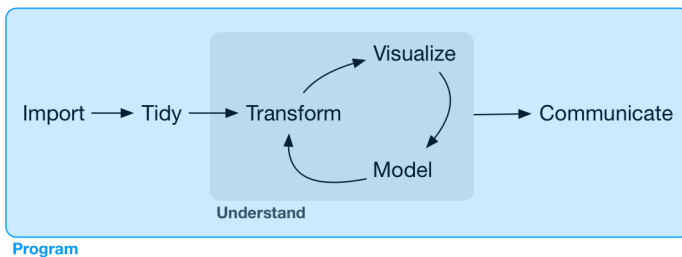
(This also illustrates another challenge of times zones: they're not all integer hour offsets!)

- Change the underlying instant in time. Use this when you have an instant that has been labeled with the incorrect time zone, and you need to fix it:

```
x4b <- force_tz(x4, tzone = "Australia/Lord_Howe")
x4b
#> [1] "2015-06-01 09:00:00 LHST"
#> [2] "2015-06-01 09:00:00 LHST"
#> [3] "2015-06-01 09:00:00 LHST"
x4b - x4
#> Time differences in hours
#> [1] -17.5 -17.5 -17.5
```

# Program

In this part of the book, you'll improve your programming skills. Programming is a cross-cutting skill needed for all data science work: you must use a computer to do data science; you cannot do it in your head, or with pencil and paper.



Programming produces code, and code is a tool of communication. Obviously code tells the computer what you want it to do. But it also communicates meaning to other humans. Thinking about code as a vehicle for communication is important because every project you do is fundamentally collaborative. Even if you're not working with other people, you'll definitely be working with future-you! Writing clear code is important so that others (like future-you) can understand why you tackled an analysis in the way you did. That means getting better at programming also involves getting better at com-

municating. Over time, you want your code to become not just easier to write, but easier for others to read.

Writing code is similar in many ways to writing prose. One parallel that I find particularly useful is that in both cases rewriting is the key to clarity. The first expression of your ideas is unlikely to be particularly clear, and you may need to rewrite multiple times. After solving a data analysis challenge, it's often worth looking at your code and thinking about whether or not it's obvious what you've done. If you spend a little time rewriting your code while the ideas are fresh, you can save a lot of time later trying to re-create what your code did. But this doesn't mean you should rewrite every function: you need to balance what you need to achieve now with saving time in the long run. (But the more you rewrite your functions the more likely your first attempt will be clear.)

In the following four chapters, you'll learn skills that will allow you to both tackle new programs and solve existing problems with greater clarity and ease:

- In [Chapter 14](#), you will dive deep into the *pipe*, `%>%`, and learn more about how it works, what the alternatives are, and when not to use it.
- Copy-and-paste is a powerful tool, but you should avoid doing it more than twice. Repeating yourself in code is dangerous because it can easily lead to errors and inconsistencies. Instead, in [Chapter 15](#), you'll learn how to write *functions*, which let you extract out repeated code so that it can be easily reused.
- As you start to write more powerful functions, you'll need a solid grounding in R's *data structures*, provided by [Chapter 16](#). You must master the four common atomic vectors and the three important S3 classes built on top of them, and understand the mysteries of the list and data frame.
- Functions extract out repeated code, but you often need to repeat the same actions on different inputs. You need tools for *iteration* that let you do similar things again and again. These tools include for loops and functional programming, which you'll learn about in [Chapter 17](#).

## Learning More

The goal of these chapters is to teach you the minimum about programming that you need to practice data science, which turns out to be a reasonable amount. Once you have mastered the material in this book, I strongly believe you should invest further in your programming skills. Learning more about programming is a long-term investment: it won't pay off immediately, but in the long term it will allow you to solve new problems more quickly, and let you reuse your insights from previous problems in new scenarios.

To learn more you need to study R as a programming language, not just an interactive environment for data science. We have written two books that will help you do so:

- *Hands-On Programming with R*, by Garrett Grolemund. This is an introduction to R as a programming language and is a great place to start if R is your first programming language. It covers similar material to these chapters, but with a different style and different motivation examples (based in the casino). It's a useful complement if you find that these four chapters go by too quickly.
- *Advanced R* by Hadley Wickham. This dives into the details of R the programming language. This is a great place to start if you have existing programming experience. It's also a great next step once you've internalized the ideas in these chapters. You can read it online at <http://adv-r.had.co.nz>.



---

# Pipes with magrittr

## Introduction

Pipes are a powerful tool for clearly expressing a sequence of multiple operations. So far, you've been using them without knowing how they work, or what the alternatives are. Now, in this chapter, it's time to explore the pipe in more detail. You'll learn the alternatives to the pipe, when you shouldn't use the pipe, and some useful related tools.

## Prerequisites

The pipe, `%>%`, comes from the **magrittr** package by Stefan Milton Bache. Packages in the tidyverse load `%>%` for you automatically, so you don't usually load **magrittr** explicitly. Here, however, we're focusing on piping, and we aren't loading any other packages, so we will load it explicitly.

```
library(magrittr)
```

## Piping Alternatives

The point of the pipe is to help you write code in a way that is easier to read and understand. To see why the pipe is so useful, we're going to explore a number of ways of writing the same code. Let's use code to tell a story about a little bunny named Foo Foo:

Little bunny Foo Foo  
Went hopping through the forest  
Scooping up the field mice  
And bopping them on the head

This is a popular children's poem that is accompanied by hand actions.

We'll start by defining an object to represent little bunny Foo Foo:

```
foo_foo <- little_bunny()
```

And we'll use a function for each key verb: `hop()`, `scoop()`, and `bop()`. Using this object and these verbs, there are (at least) four ways we could retell the story in code:

- Save each intermediate step as a new object.
- Overwrite the original object many times.
- Compose functions.
- Use the pipe.

We'll work through each approach, showing you the code and talking about the advantages and disadvantages.

## Intermediate Steps

The simplest approach is to save each step as a new object:

```
foo_foo_1 <- hop(foo_foo, through = forest)
foo_foo_2 <- scoop(foo_foo_1, up = field_mice)
foo_foo_3 <- bop(foo_foo_2, on = head)
```

The main downside of this form is that it forces you to name each intermediate element. If there are natural names, this is a good idea, and you should do it. But many times, like in this example, there aren't natural names, and you add numeric suffixes to make the names unique. That leads to two problems:

- The code is cluttered with unimportant names.
- You have to carefully increment the suffix on each line.

Whenever I write code like this, I invariably use the wrong number on one line and then spend 10 minutes scratching my head and trying to figure out what went wrong with my code.



You may also worry that this form creates many copies of your data and takes up a lot of memory. Surprisingly, that's not the case. First, note that proactively worrying about memory is not a useful way to spend your time: worry about it when it becomes a problem (i.e., you run out of memory), not before. Second, R isn't stupid, and it will share columns across data frames, where possible. Let's take a look at an actual data manipulation pipeline where we add a new column to `ggplot2::diamonds`:

```
diamonds <- ggplot2::diamonds
diamonds2 <- diamonds %>%
  dplyr::mutate(price_per_carat = price / carat)

pryr::object_size(diamonds)
#> 3.46 MB
pryr::object_size(diamonds2)
#> 3.89 MB
pryr::object_size(diamonds, diamonds2)
#> 3.89 MB
```

`pryr::object_size()` gives the memory occupied by all of its arguments. The results seem counterintuitive at first:

- `diamonds` takes up 3.46 MB.
- `diamonds2` takes up 3.89 MB.
- `diamonds` and `diamonds2` together take up 3.89 MB!

How can that work? Well, `diamonds2` has 10 columns in common with `diamonds`: there's no need to duplicate all that data, so the two data frames have variables in common. These variables will only get copied if you modify one of them. In the following example, we modify a single value in `diamonds$carat`. That means the `carat` variable can no longer be shared between the two data frames, and a copy must be made. The size of each data frame is unchanged, but the collective size increases:

```
diamonds$carat[1] <- NA
pryr::object_size(diamonds)
#> 3.46 MB
pryr::object_size(diamonds2)
#> 3.89 MB
pryr::object_size(diamonds, diamonds2)
#> 4.32 MB
```

(Note that we use `pryr::object_size()` here, not the built-in `object.size()`. `object.size()` only takes a single object so it can't compute how data is shared across multiple objects.)

## Overwrite the Original

Instead of creating intermediate objects at each step, we could overwrite the original object:

```
foo_foo <- hop(foo_foo, through = forest)
foo_foo <- scoop(foo_foo, up = field_mice)
foo_foo <- bop(foo_foo, on = head)
```

This is less typing (and less thinking), so you're less likely to make mistakes. However, there are two problems:

- Debugging is painful. If you make a mistake you'll need to re-run the complete pipeline from the beginning.
- The repetition of the object being transformed (we've written `foo_foo` six times!) obscures what's changing on each line.

## Function Composition

Another approach is to abandon assignment and just string the function calls together:

```
bop(
  scoop(
    hop(foo_foo, through = forest),
    up = field_mice
  ),
  on = head
)
```

Here the disadvantage is that you have to read from inside-out, from right-to-left, and that the arguments end up spread far apart (evocatively called the **Dagwood sandwich** problem). In short, this code is hard for a human to consume.

## Use the Pipe

Finally, we can use the pipe:

```
foo_foo %>%
  hop(through = forest) %>%
  scoop(up = field_mouse) %>%
  bop(on = head)
```

This is my favorite form, because it focuses on verbs, not nouns. You can read this series of function compositions like it's a set of imperative actions. Foo Foo hops, then scoops, then bops. The downside, of course, is that you need to be familiar with the pipe. If you've never seen `%>%` before, you'll have no idea what this code does. Fortunately, most people pick up the idea very quickly, so when you share your code with others who aren't familiar with the pipe, you can easily teach them.

The pipe works by performing a “lexical transformation”: behind the scenes, **magrittr** reassembles the code in the pipe to a form that works by overwriting an intermediate object. When you run a pipe like the preceding one, **magrittr** does something like this:

```
my_pipe <- function(.) {  
  . <- hop(., through = forest)  
  . <- scoop(., up = field_mice)  
  bop(., on = head)  
}  
my_pipe(foo_foo)
```

This means that the pipe won't work for two classes of functions:

- Functions that use the current environment. For example, `assign()` will create a new variable with the given name in the current environment:

```
assign("x", 10)  
x  
#> [1] 10  
  
"x" %>% assign(100)  
x  
#> [1] 10
```

The use of `assign` with the pipe does not work because it assigns it to a temporary environment used by `%>%`. If you do want to use `assign` with the pipe, you must be explicit about the environment:

```
env <- environment()  
"x" %>% assign(100, envir = env)  
x  
#> [1] 100
```

Other functions with this problem include `get()` and `load()`.

- Functions that use lazy evaluation. In R, function arguments are only computed when the function uses them, not prior to call-

ing the function. The pipe computes each element in turn, so you can't rely on this behavior.

One place that this is a problem is `tryCatch()`, which lets you capture and handle errors:

```
tryCatch(stop("!"), error = function(e) "An error")
#> [1] "An error"

stop("!") %>%
  tryCatch(error = function(e) "An error")
#> Error in eval(expr, envir, enclos): !
```

There are a relatively wide class of functions with this behavior, including `try()`, `suppressMessages()`, and `suppressWarnings()` in base R.

## When Not to Use the Pipe

The pipe is a powerful tool, but it's not the only tool at your disposal, and it doesn't solve every problem! Pipes are most useful for rewriting a fairly short linear sequence of operations. I think you should reach for another tool when:

- Your pipes are longer than (say) 10 steps. In that case, create intermediate objects with meaningful names. That will make debugging easier, because you can more easily check the intermediate results, and it makes it easier to understand your code, because the variable names can help communicate intent.
- You have multiple inputs or outputs. If there isn't one primary object being transformed, but two or more objects being combined together, don't use the pipe.
- You are starting to think about a directed graph with a complex dependency structure. Pipes are fundamentally linear and expressing complex relationships with them will typically yield confusing code.

## Other Tools from **magrittr**

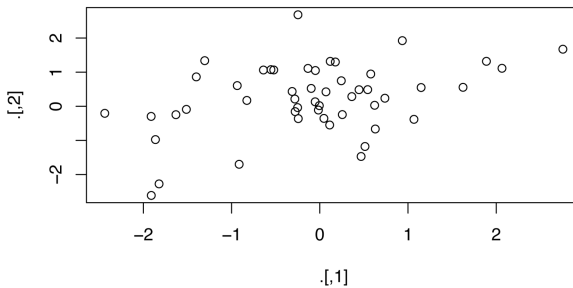
All packages in the tidyverse automatically make `%>%` available for you, so you don't normally load **magrittr** explicitly. However, there

are some other useful tools inside **magrittr** that you might want to try out:

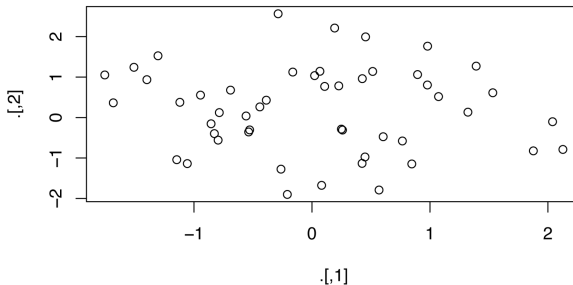
- When working with more complex pipes, it's sometimes useful to call a function for its side effects. Maybe you want to print out the current object, or plot it, or save it to disk. Many times, such functions don't return anything, effectively terminating the pipe.

To work around this problem, you can use the “tee” pipe. `%T>%` works like `%>%` except that it returns the lefthand side instead of the righthand side. It's called “tee” because it's like a literal T-shaped pipe:

```
rnorm(100) %>%  
  matrix(ncol = 2) %>%  
  plot() %>%  
  str()  
#> NULL
```



```
rnorm(100) %>%  
  matrix(ncol = 2) %T>%  
  plot() %>%  
  str()  
#> num [1:50, 1:2] -0.387 -0.785 -1.057 -0.796 -1.756 ...
```



- If you're working with functions that don't have a data frame-based API (i.e., you pass them individual vectors, not a data frame and expressions to be evaluated in the context of that data frame), you might find `%%` useful. It “explodes” out the variables in a data frame so that you can refer to them explicitly. This is useful when working with many functions in base R:

```
mtcars %$%  
  cor(displ, mpg)  
#> [1] -0.848
```

- For assignment **magrittr** provides the `%<>%` operator, which allows you to replace code like:

```
mtcars <- mtcars %>%  
  transform(cyl = cyl * 2)
```

with:

```
mtcars %<>% transform(cyl = cyl * 2)
```

I'm not a fan of this operator because I think assignment is such a special operation that it should always be clear when it's occurring. In my opinion, a little bit of duplication (i.e., repeating the name of the object twice) is fine in return for making assignment more explicit.

# Functions

## Introduction

One of the best ways to improve your reach as a data scientist is to write functions. Functions allow you to automate common tasks in a more powerful and general way than copying and pasting. Writing a function has three big advantages over using copy-and-paste:

- You can give a function an evocative name that makes your code easier to understand.
- As requirements change, you only need to update code in one place, instead of many.
- You eliminate the chance of making incidental mistakes when you copy and paste (i.e., updating a variable name in one place, but not in another).

Writing good functions is a lifetime journey. Even after using R for many years I still learn new techniques and better ways of approaching old problems. The goal of this chapter is not to teach you every esoteric detail of functions but to get you started with some pragmatic advice that you can apply immediately.

As well as practical advice for writing functions, this chapter also gives you some suggestions for how to style your code. Good code style is like correct punctuation. You can manage without it, but it sure makes things easier to read! As with styles of punctuation, there are many possible variations. Here we present the style we use in our code, but the most important thing is to be consistent.

## Prerequisites

The focus of this chapter is on writing functions in base R, so you won't need any extra packages.

## When Should You Write a Function?

You should consider writing a function whenever you've copied and pasted a block of code more than twice (i.e., you now have three copies of the same code). For example, take a look at this code. What does it do?

```
df <- tibble::tibble(  
  a = rnorm(10),  
  b = rnorm(10),  
  c = rnorm(10),  
  d = rnorm(10)  
)  
  
df$a <- (df$a - min(df$a, na.rm = TRUE)) /  
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))  
df$b <- (df$b - min(df$b, na.rm = TRUE)) /  
  (max(df$b, na.rm = TRUE) - min(df$a, na.rm = TRUE))  
df$c <- (df$c - min(df$c, na.rm = TRUE)) /  
  (max(df$c, na.rm = TRUE) - min(df$c, na.rm = TRUE))  
df$d <- (df$d - min(df$d, na.rm = TRUE)) /  
  (max(df$d, na.rm = TRUE) - min(df$d, na.rm = TRUE))
```

You might be able to puzzle out that this rescales each column to have a range from 0 to 1. But did you spot the mistake? I made an error when copying and pasting the code for `df$b`: I forgot to change an `a` to a `b`. Extracting repeated code out into a function is a good idea because it prevents you from making this type of mistake.

To write a function you need to first analyze the code. How many inputs does it have?

```
(df$a - min(df$a, na.rm = TRUE)) /  
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))
```

This code only has one input: `df$a`. (If you're surprised that `TRUE` is not an input, you can explore why in the following exercise.) To make the inputs more clear, it's a good idea to rewrite the code using temporary variables with general names. Here this code only requires a single numeric vector, so I'll call it `x`:

```
x <- df$a  
(x - min(x, na.rm = TRUE)) /
```



```
(max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
#> [1] 0.289 0.751 0.000 0.678 0.853 1.000 0.172 0.611 0.612
#> [10] 0.601
```

There is some duplication in this code. We're computing the range of the data three times, but it makes sense to do it in one step:

```
rng <- range(x, na.rm = TRUE)
(x - rng[1]) / (rng[2] - rng[1])
#> [1] 0.289 0.751 0.000 0.678 0.853 1.000 0.172 0.611 0.612
#> [10] 0.601
```

Pulling out intermediate calculations into named variables is a good practice because it makes it more clear what the code is doing. Now that I've simplified the code, and checked that it still works, I can turn it into a function:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(c(0, 5, 10))
#> [1] 0.0 0.5 1.0
```

There are three key steps to creating a new function:

1. You need to pick a *name* for the function. Here I've used `rescale01` because this function rescales a vector to lie between 0 and 1.
2. You list the inputs, or *arguments*, to the function inside `function`. Here we have just one argument. If we had more the call would look like `function(x, y, z)`.
3. You place the code you have developed in the *body* of the function, a `{` block that immediately follows `function(...)`.

Note the overall process: I only made the function after I'd figured out how to make it work with a simple input. It's easier to start with working code and turn it into a function; it's harder to create a function and then try to make it work.

At this point it's a good idea to check your function with a few different inputs:

```
rescale01(c(-10, 0, 10))
#> [1] 0.0 0.5 1.0
rescale01(c(1, 2, 3, NA, 5))
#> [1] 0.00 0.25 0.50 NA 1.00
```

As you write more and more functions you'll eventually want to convert these informal, interactive tests into formal, automated tests. That process is called unit testing. Unfortunately, it's beyond the scope of this book, but you can learn about it at <http://r-pkgs.had.co.nz/tests.html>.

We can simplify the original example now that we have a function:

```
df$a <- rescale01(df$a)
df$b <- rescale01(df$b)
df$c <- rescale01(df$c)
df$d <- rescale01(df$d)
```

Compared to the original, this code is easier to understand and we've eliminated one class of copy-and-paste errors. There is still quite a bit of duplication since we're doing the same thing to multiple columns. We'll learn how to eliminate that duplication in [Chapter 17](#), once you've learned more about R's data structures in [Chapter 16](#).

Another advantage of functions is that if our requirements change, we only need to make the change in one place. For example, we might discover that some of our variables include infinite values, and `rescale01()` fails:

```
x <- c(1:10, Inf)
rescale01(x)
#> [1] 0 0 0 0 0 0 0 0 0 0 NaN
```

Because we've extracted the code into a function, we only need to make the fix in one place:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(x)
#> [1] 0.000 0.111 0.222 0.333 0.444 0.556 0.667 0.778 0.889
#> [10] 1.000 Inf
```

This is an important part of the “do not repeat yourself” (or DRY) principle. The more repetition you have in your code, the more places you need to remember to update when things change (and they always do!), and the more likely you are to create bugs over time.

## Exercises

1. Why is `TRUE` not a parameter to `rescale01()`? What would happen if `x` contained a single missing value, and `na.rm` was `FALSE`?
2. In the second variant of `rescale01()`, infinite values are left unchanged. Rewrite `rescale01()` so that `-Inf` is mapped to 0, and `Inf` is mapped to 1.
3. Practice turning the following code snippets into functions. Think about what each function does. What would you call it? How many arguments does it need? Can you rewrite it to be more expressive or less duplicative?

```
mean(is.na(x))
```

```
x / sum(x, na.rm = TRUE)
```

```
sd(x, na.rm = TRUE) / mean(x, na.rm = TRUE)
```

4. Follow <http://nicercode.github.io/intro/writing-functions.html> to write your own functions to compute the variance and skew of a numeric vector.
5. Write `both_na()`, a function that takes two vectors of the same length and returns the number of positions that have an NA in both vectors.
6. What do the following functions do? Why are they useful even though they are so short?

```
is_directory <- function(x) file.info(x)$isdir  
is_readable <- function(x) file.access(x, 4) == 0
```

7. Read the [complete lyrics](#) to “Little Bunny Foo Foo.” There’s a lot of duplication in this song. Extend the initial piping example to re-create the complete song, and use functions to reduce the duplication.

## Functions Are for Humans and Computers

It’s important to remember that functions are not just for the computer, but are also for humans. R doesn’t care what your function is called, or what comments it contains, but these are important for human readers. This section discusses some things that you should bear in mind when writing functions that humans can understand.

The name of a function is important. Ideally, the name of your function will be short, but clearly evoke what the function does. That's hard! But it's better to be clear than short, as RStudio's autocomplete makes it easy to type long names.

Generally, function names should be verbs, and arguments should be nouns. There are some exceptions: nouns are OK if the function computes a very well known noun (i.e., `mean()` is better than `compute_mean()`), or is accessing some property of an object (i.e., `coef()` is better than `get_coefficients()`). A good sign that a noun might be a better choice is if you're using a very broad verb like "get," "compute," "calculate," or "determine." Use your best judgment and don't be afraid to rename a function if you figure out a better name later:

```
# Too short
f()

# Not a verb, or descriptive
my_awesome_function()

# Long, but clear
impute_missing()
collapse_years()
```

If your function name is composed of multiple words, I recommend using "snake\_case," where each lowercase word is separated by an underscore. camelCase is a popular alternative. It doesn't really matter which one you pick; the important thing is to be consistent: pick one or the other and stick with it. R itself is not very consistent, but there's nothing you can do about that. Make sure you don't fall into the same trap by making your code as consistent as possible:

```
# Never do this!
col_mins <- function(x, y) {}
rowMaxes <- function(y, x) {}
```

If you have a family of functions that do similar things, make sure they have consistent names and arguments. Use a common prefix to indicate that they are connected. That's better than a common suffix because autocomplete allows you to type the prefix and see all the members of the family:

```
# Good
input_select()
input_checkbox()
input_text()
```